

第 3 章：关系数据库标准语言 SQL

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

Hello World

```
SELECT 'Hello World';
```

教学内容¹

- 1 3.1 SQL 标准数据类型
- 2 3.2 SQL 数据定义
 - 3.2.1 创建表
 - 3.2.2 删除表
 - 3.2.3 修改表模式
- 3 3.3 SQL 数据查询
 - 3.3.1 单表查询
 - 3.3.2 集合查询
 - 3.3.3 分组查询
 - 3.3.4 连接查询
 - 3.3.5 嵌套查询
- 4 3.4 SQL 数据更新
 - 3.4.1 插入元组
 - 3.4.2 删除元组
 - 3.4.3 修改元组
 - 3.4.4 完整性约束检查
- 5 3.5 视图

3.1 SQL 标准数据类型

SQL 标准数据类型

数值型

- INT或INTEGER: 整数, 取值范围取决于 DBMS 实现
- SMALLINT: 整数, 取值范围比INT小
- BIGINT: 整数, 取值范围比INT大
- NUMERIC(p , s)或DECIMAL(p , s): 定点数, p 位有效数字, 小数点后 s 位
- FLOAT(n): 浮点数, 精度至少为 n 位数字
- REAL: 同FLOAT, 但精度由 DBMS 确定
- DOUBLE PRECISION: 同FLOAT, 但精度由 DBMS 确定, 比REAL高

布尔型

- BOOLEAN: 真值TRUE或FALSE

SQL 基本数据类型 (续)

字符串型

- CHAR(n)或CHARACTER(n): 定长字符串, 长度为 n
- VARCHAR(n)或CHARACTER VARYING(n): 变长字符串, 最大长度为 n
- CLOB或CHARACTER LARGE OBJECT: 超长字符串, 长度超过VARCHAR(n)类型字符串的长度上限

二进制串型

- BINARY(n): 定长二进制串, 长度为 n 个字节
- VARBINARY(n): 变长二进制串, 最大长度为 n 个字节
- BLOB或BINARY LARGE OBJECT: 超长二进制串, 长度超过VARBINARY(n)类型二进制串的长度上限

SQL 基本数据类型

日期时间型

- DATE: 日期, 格式YYYY-MM-DD
- TIME: 时间, 格式HH:MM:SS
- TIME WITH TIME ZONE: 时间, 包含时区
- TIMESTAMP: 日期和时间, 格式YYYY-MM-DD HH:MM:SS
- TIMESTAMP WITH TIME ZONE: 日期和时间, 包含时区

时间区间型

- INTERVAL YEAR TO MONTH: 时间区间, 用年和月表示
- INTERVAL DAY TO SECOND: 时间区间, 用日、时、分、秒表示

3.2 SQL 数据定义

元组的多重集合 (Multi-Set)

- 创建表
- 删除表
- 修改表模式

3.2.1 创建表

创建表

CREATE TABLE

- 定义表名、属性名、属性类型、主键、外键、完整性约束

例 (创建表)

创建 Student 表，其模式如下：

属性名	属性类型	属性含义
Sno	CHAR(6)	学号
Sname	VARCHAR(10)	姓名
Ssex	CHAR	性别
Sage	INT	年龄
Sdept	VARCHAR(20)	所在系

```
CREATE TABLE Student (  
  Sno CHAR(6),  
  Sname VARCHAR(10),  
  Ssex CHAR,  
  Sage INT,  
  Sdept VARCHAR(20)  
);
```

声明主键

PRIMARY KEY

例 (声明主键)

❶ 声明 Student 表的主键 (Sno)

通用方法:

```
CREATE TABLE Student (  
    Sno CHAR(6),  
    Sname VARCHAR(10),  
    Ssex CHAR,  
    Sage INT,  
    Sdept VARCHAR(20),  
    PRIMARY KEY (Sno)  
);
```

简便方法 (仅限单属性主键):

```
CREATE TABLE Student (  
    Sno CHAR(6) PRIMARY KEY,  
    Sname VARCHAR(10),  
    Ssex CHAR,  
    Sage INT,  
    Sdept VARCHAR(20)  
);
```

声明外键

FOREIGN KEY

例 (声明外键)

❶ 创建 SC 表, 其模式如下, 其中 (Sno, Cno) 是主键, (Sno) 是外键, 参照 Student 的主键 (Sno)

属性名	属性类型	属性含义
Sno	CHAR(6)	学号
Cno	CHAR(4)	课号
Grade	INT	成绩

通用方法:

```
CREATE TABLE SC (  
    Sno CHAR(6), Cno CHAR(4), Grade  
    INT,  
    PRIMARY KEY (Sno, Cno),  
    FOREIGN KEY (Sno) REFERENCES  
    Student (Sno)  
);
```

简便方法 (仅限单属性外键):

```
CREATE TABLE SC (  
    Sno CHAR(6) REFERENCES Student (  
    Sno),  
    Cno CHAR(4), Grade INT,  
    PRIMARY KEY (Sno, Cno)  
);
```

声明用户定义完整性约束

- 规定属性值非空: NOT NULL
- 规定属性值不重复: UNIQUE
- 规定属性的缺省值: DEFAULT 缺省值
- 规定属性值必须满足表达式给出的约束条件: CHECK (表达式)
- 规定数值型属性的值自动递增: GENERATED AS IDENTITY

例 (声明用户定义完整性约束)

```
CREATE TABLE Student (  
    Sno CHAR(6),  
    Sname VARCHAR(10) NOT NULL,  
    Ssex CHAR NOT NULL CHECK (Ssex IN ('M', 'F')),  
    Sage INT DEFAULT 0 CHECK (Sage >= 0),  
    Sdept VARCHAR(20),  
    PRIMARY KEY (Sno)  
);
```

3.2.2 删除表

删除表

DROP TABLE

例 (删除表)

删除 Student 表:

```
DROP TABLE Student;
```

3.2.3 修改表模式

修改表模式

ALTER TABLE

- 修改表名、属性名
- 添加、修改、删除属性
- 添加、删除完整性约束

改名

例 (修改表名)

将 Student 表改名为 XueSheng

```
ALTER TABLE Student RENAME TO XueSheng;
```

例 (修改属性名)

将 Student 表的 Sno 属性改名为 Sid

```
ALTER TABLE Student RENAME COLUMN Sno TO Sid;
```

添加/删除/修改属性

例 (添加属性)

在 Student 表中添加属性 Mno, 记录学生的班长的学号

```
ALTER TABLE Student ADD COLUMN Mno CHAR(6);
```

例 (删除属性)

删除 Student 表的 Mno 属性

```
ALTER TABLE Student DROP COLUMN Mno;
```

例 (修改属性类型)

修改 Student 表的 Mno 属性的类型

```
ALTER TABLE Student ALTER COLUMN Sno TYPE VARCHAR(6);
```

添加/删除属性非空约束

例 (添加属性非空约束)

给 Student 表的 Sname 属性添加非空约束

```
ALTER TABLE Student ALTER COLUMN SET NOT NULL;
```

例 (删除属性非空约束)

删除 Student 表的 Sname 属性的非空约束

```
ALTER TABLE Student ALTER COLUMN DROP NOT NULL;
```

添加/删除主键

例 (添加主键)

将 (Sno) 声明为 Student 表的主键

```
ALTER TABLE Student ADD PRIMARY KEY (Sno);
```

例 (删除主键)

删除 Student 表的主键

```
ALTER TABLE Student DROP PRIMARY KEY;
```

不同 DBMS 使用的语法可能不同, 甚至不支持

添加/删除外键

例 (添加外键)

将 (Mno) 声明为 Student 表的外键, 参照 Student 的主键 (Sno)

```
ALTER TABLE Student ADD CONSTRAINT fk_mno  
FOREIGN KEY (Mno) REFERENCES Student (Sno);
```

例 (删除外键)

删除 Student 表上的外键约束 fk_mno

```
ALTER TABLE Student DROP CONSTRAINT fk_mno;
```

不同 DBMS 使用的语法可能不同, 甚至不支持

探究式学习：RDBMS 的 DDL 方言

历史遗留问题

- 查阅 RDBMS 的文档
- 询问 LLM

3.3 SQL 数据查询

SQL 数据查询

SELECT

- 单表查询
- 集合查询
- 分组查询
- 连接查询
- 嵌套查询

3.3.1 单表查询

投影查询

SELECT 属性列表 FROM 表名

- 若要返回全部列，可将属性列表简写为*

例 (投影查询)

- 查询全体学生的学号和姓名
| `SELECT Sno, Sname FROM Student;`
- 查询所有系名
| `SELECT Sdept FROM Student;`
- 查询全体学生的全部信息
| `SELECT * FROM Student;`

查询结果去重

SELECT DISTINCT

例 (查询结果去重)

- 查询所有不同的系名
| `SELECT DISTINCT Sdept FROM Student;`

扩展的投影查询

SELECT 表达式列表 FROM 表名

- 表达式：常量、属性名、算术表达式、函数表达式、逻辑表达式
- 查询结果中列的名称就是对应表达式本身

例 (扩展的投影查询)

- 查询全体学生的学号和姓名 (姓名全大写)
| `SELECT Sno, UPPER(Sname) FROM Student;`
- 查询全体学生的姓名和出生年份
| `SELECT Sname, 2026 - Sage FROM Student;`

属性重命名

表达式 AS 名称

- 更名后不可再用原名

例 (属性重命名)

- 查询全体学生的学号和姓名 (姓名全大写)
| `SELECT Sno, UPPER(Sname) AS UpperName FROM Student;`
- 查询全体学生的姓名和出生年份
| `SELECT Sname, 2026 - Sage AS BirthYear FROM Student;`

表重命名

表名 AS 新表名

- 更名后不可再用原名

例 (表重命名)

- 查询全体学生的学号和姓名

```
SELECT S.Sno, S.Sname FROM Student AS S;
```

选择查询

SELECT ... FROM ... WHERE 选择条件

例 (选择查询)

- 查询计算机系 (CS) 全体学生的学号和姓名

```
SELECT Sno, Sname FROM Student WHERE Sdept = 'CS';
```

- 查询计算机系 (CS) 全体男同学的学号和姓名

```
SELECT Sno, Sname FROM Student WHERE Sdept = 'CS' AND Ssex = 'M';
```

- 查询计算机系 (CS) 和数学系 (Math) 全体学生的学号和姓名

```
SELECT Sno, Sname FROM Student WHERE Sdept = 'CS' OR Sdept = 'Math';
```

选择条件

表达式比较

- 语法：表达式 1 比较运算符 表达式 2
- 比较运算符：=, <, <=, >, >=, !=, <>

范围比较

- 语法：表达式 1 [NOT] BETWEEN 表达式 2 AND 表达式 3
- 功能：判断表达式 1 的值是否 (不) 在表达式 2 和表达式 3 之间

集合元素判断

- 语法：表达式 1 [NOT] IN (表达式 2, ..., 表达式 n)
- 功能：判断表达式 1 的值是否 (不) 在表达式 2, ..., 表达式 n 的值构成的集合中

选择查询条件

字符串匹配

- 语法：字符串表达式 [NOT] LIKE 模式 [ESCAPE 转义字符]
- 功能：判断字符串表达式的值是否匹配给定的含有通配符的模式
 - 通配符_：匹配单个字符
 - 通配符%：匹配任意长度的字符串 (含空串)
 - 可使用 ESCAPE 指定转义字符，默认转义字符为 \

例 (字符串匹配)

- 查询姓名首字母为 E 的学生的学号和姓名

```
SELECT Sno, Sname FROM Student WHERE Sname LIKE 'E%';
```

- 查询姓名为 4 个字母且首字母为 E 的学生的学号和姓名

```
SELECT Sno, Sname FROM Student WHERE Sname LIKE 'E____';
```

选择查询条件

空值判断

- 语法: 属性名 IS [NOT] NULL
- 功能: 判断属性值是否 (不) 为空
- 错误写法: 属性名 = NULL, 属性名 <> NULL, 属性名 != NULL。这些错误写法的结果均为 UNKNOWN (既非真, 也非假)

例 (空值判断)

- ❶ 查询选了课但还未取得成绩的学生

```
SELECT Sno FROM SC WHERE Grade IS NULL;
```

- ❷ 下面的 SQL 语句的结果是什么?

```
SELECT Sno FROM SC WHERE Grade = NULL;
```

选择查询条件

逻辑运算

- 逻辑运算符: AND, OR, NOT

- 逻辑运算真值表

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

NOT	TRUE	FALSE	UNKNOWN
	FALSE	TRUE	UNKNOWN

- 只有使查询条件为真的元组才能出现在查询结果中
- UNKNOWN 只能表示逻辑运算结果, 不能用作布尔型属性的值

查询结果排序

ORDER BY 排序属性列表

- 放在查询语句末尾
- 按排序属性的字典序排列元组
- ASC 表示升序排列, DESC 表示降序排列
- 每个排序属性默认按属性值升序排列, 空值排在非空值前面

例 (集合交查询)

- ❶ 查询计算机系全体学生的学号和姓名, 并按学号升序排列

```
SELECT Sno, Sname FROM Student WHERE Sdept = 'CS' ORDER BY Sno;
```

- ❷ 查询全体学生的信息, 结果按所在系升序排列, 同一个系的学生按年龄降序排列

```
SELECT * FROM Student ORDER BY Sdept ASC, Sage DESC;
```

限制查询结果数量

LIMIT 结果数量 [OFFSET 偏移量]

- 放在查询语句末尾
- 从偏移量位置 (默认是 0) 的元组开始, 返回指定数量的元组

例 (限制查询结果数量)

- ❶ 查询 3006 号课得分最高的前 2 名学生的学号和成绩

```
SELECT Sno, Grade FROM SC WHERE Cno = '3006'  
ORDER BY Grade DESC LIMIT 2;
```

LIMIT 不是标准 SQL 的内容

习题：单表查询

使用课程提供的product-db.sql在 RDBMS 上创建 Products 数据库 (Database Systems The Complete Book – Exercise 2.4.1), 并使用 SQL 完成下列查询

SELECT ... FROM ...

- 1 What are the manufacturers?
- 2 Find the model numbers and the hard disk size (in GB) of all PC's

SELECT ... FROM ... WHERE ...

- 1 What PC models have a speed of at least 3.00 and ram of at least 1024MB?
- 2 What PC models have a speed of at least 3.00 or ram of at least 1024MB?
- 3 What models does the manufacturer A produce?
- 4 Find the model numbers of all color laser printers

3.3.2 集合查询

集合并查询

查询1 UNION 查询2

例 (集合并查询)

- 1 查询选修了 1002 或 3006 号课的学生的学号

```
SELECT Sno FROM SC WHERE Cno = '1002' UNION
SELECT Sno FROM SC WHERE Cno = '3006';
```

查询结果合并 (不去重)

查询1 UNION ALL 查询2

例 (集合并查询)

- 1 查询选修了 1002 或 3006 号课的学生的学号

```
SELECT Sno FROM SC WHERE Cno = '1002' UNION ALL
SELECT Sno FROM SC WHERE Cno = '3006';
```

集合交查询

查询1 INTERSECT 查询2

例 (集合交查询)

① 查询选修了 1002 和 3006 号课的学生的学号

```
SELECT Sno FROM SC WHERE Cno = '1002' INTERSECT
SELECT Sno FROM SC WHERE Cno = '3006';
```

不是所有 RDBMS 都支持INTERSECT (如 MySQL 8.0.31 以下版本)

集合差查询

查询1 EXCEPT 查询2

例 (集合交查询)

① 查询选修了 1002 号课, 但没选修 3006 号课的学生的学号

```
SELECT Sno FROM SC WHERE Cno = '1002' EXCEPT
SELECT Sno FROM SC WHERE Cno = '3006';
```

不是所有 RDBMS 都支持EXCEPT (如 MySQL 8.0.31 以下版本)

探究式学习：集合查询

借助 LLM 学习

- INTERSECT ALL和EXCEPT ALL是什么意思?
- MySQL 8.0.31 以下版本如何实现INTERSECT和EXCEPT?

习题：集合查询

UNION

- ① Find the model numbers and price of all PC's and all laptops
- ② What manufacturers make all types of products (PC, laptop, and printer)?

3.3.3 分组查询

聚集函数

SELECT agg(E) FROM R

例 (聚集函数)

- ❶ 查询计算机系全体学生的数量
| SELECT COUNT(*) FROM Student WHERE Sdept = 'CS';
- ❷ 查询计算机系学生的最大年龄
| SELECT MAX(Sage) FROM Student WHERE Sdept = 'CS';

聚集函数

SELECT agg(E) FROM R

- 用聚集函数 agg 求关系 R 上某表达式 E 的所有结果的聚集值
- COUNT()计数, MAX()求最大值, MIN()求最小值, SUM()求和, AVG()求平均值

COUNT(*)	所有元组的数量
COUNT(表达式)	非空表达式值的数量
COUNT(DISTINCT 表达式)	不同的非空表达式值的数量
MAX(表达式)	表达式的最大值
MIN(表达式)	表达式的最小值
SUM(表达式)	表达式值的和
SUM(DISTINCT 表达式)	不同的表达式值的和
AVG(表达式)	表达式值的平均数
AVG(DISTINCT 表达式)	不同的表达式值的平均数

聚集函数

聚集函数不能出现在WHERE 子句中

例 (聚集函数)

- ❶ 查询年龄最大的学生的学号
| SELECT Sno FROM Student WHERE Sage = MAX(Sage);
写法错误, 正确的做法是使用嵌套查询 (3.3.5 节)

分组查询

GROUP BY

SELECT 分组属性列表, 聚合函数表达式列表 **FROM** 关系名 **WHERE** 选择条件
GROUP BY 分组属性列表

- ① 根据分组属性, 将关系中的元组分组, 分组属性值相同的元组分为一组
- ② 在每个分组上计算全部聚合函数表达式的结果

例 (分组查询)

- ① 统计每门课的选课人数和平均成绩

```
SELECT Cno, COUNT(*), AVG(Grade) FROM SC GROUP BY Cno;
```

- ② 统计每个系的男生人数和女生人数

```
SELECT Sdept, Ssex, COUNT(*) FROM Student GROUP BY Sdept, Ssex;
```

SELECT子句中不能包含分组属性及聚合函数表达式以外的其他表达式。(为什么?)

分组筛选

GROUP BY ... HAVING ...

SELECT 分组属性列表, 聚合函数表达式列表 **FROM** 关系名 **WHERE** 选择条件
GROUP BY 分组属性列表 **HAVING** 分组筛选条件

完成分组后, 根据分组内元组的统计信息对分组进行筛选

例 (分组筛选)

- ① 查询选修了 2 门以上课程的学生的学号和选课数

```
SELECT Sno, COUNT(*) FROM SC  
GROUP BY Sno HAVING COUNT(*) >= 2;
```

- ② 查询 2 门以上课程得分超过 80 的学生的学号及这些课程的平均分

```
SELECT Sno, AVG(Grade) FROM SC WHERE Grade >= 80  
GROUP BY Sno HAVING COUNT(*) >= 2;
```

分组查询的执行过程

选择元组 → 建立分组 → 分组筛选 → 聚集计算

探究式学习: 窗口函数 (Window Function)

借助 LLM 学习

- 什么是窗口函数?
- 窗口函数与聚合函数有什么区别?
- 什么情况下使用窗口函数?

例 (窗口函数)

查询每名已选课学生的单科成绩以及在课程中的排名

Sno	Cno	Grade	Rank
PH-001	1002	92	2
PH-001	2003	85	1
PH-001	3006	88	2
CS-001	1002	95	1
CS-001	3006	90	1
CS-002	3006	80	3
MA-001	1002	91	3

习题：分组查询

`SELECT ... FROM ... GROUP BY ...`

- ① How many models does every manufacturer have?
- ② How many models does every manufacturer have for every type of products?

`SELECT ... FROM ... GROUP BY ... HAVING ...`

- ① Find those hard-disk sizes that occur in two or more PC's
- ② What manufacturers make all types of products (PC, laptop, and printer)?

`SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ...`

- ① Find those hard-disk sizes of at least 100GB that occur in two or more PC's

3.3.4 连接查询

笛卡尔积

`SELECT ... FROM 关系1 CROSS JOIN 关系2`

例 (笛卡尔积)

- ① 查询学生及其选课情况，列出学号、姓名、课号、得分

```
SELECT Student.Sno, Sname, Cno, Grade
FROM Student CROSS JOIN SC
WHERE Student.Sno = SC.Sno;
```

内连接

`关系1 JOIN 关系2 ON 连接条件`

例 (内连接)

- ① 查询学生及其选课情况，列出学号、姓名、课号、得分

```
SELECT Student.Sno, Sname, Cno, Grade
FROM Student JOIN SC ON Student.Sno = SC.Sno;
```

同名属性等值内连接

R JOIN S USING (同名等值连接属性列表)

- 当内连接是等值连接，且连接属性同名时，可使用该语法
- 每个同名连接属性只保留一个副本

例 (同名属性等值内连接)

- ① 查询学生及其选课情况，列出学号、姓名、课号、得分

```
SELECT Student.Sno, Sname, Cno, Grade
FROM Student JOIN SC USING (Sno);
```

自然连接

关系1 NATURAL JOIN 关系2

例 (自然连接)

- ① 查询学生及其选课情况，列出学号、姓名、课号、得分

```
SELECT Student.Sno, Sname, Cno, Grade
FROM Student NATURAL JOIN SC;
```

外连接

- 左外连接：关系1 LEFT OUTER JOIN 关系2 ON 连接条件
- 右外连接：关系1 RIGHT OUTER JOIN 关系2 ON 连接条件
- 全外连接：关系1 FULL OUTER JOIN 关系2 ON 连接条件

例 (左外连接)

- ① 查询没有选课的学生的学号和姓名

```
SELECT Student.Sno, Sname
FROM Student LEFT OUTER JOIN SC ON Student.Sno = SC.Sno
WHERE Cno IS NULL;
```

同名属性等值外连接

当外连接是等值连接，且连接属性同名时，可使用如下语法

- 左外连接：关系1 LEFT OUTER JOIN 关系2 USING (同名等值连接属性列表)
- 右外连接：关系1 RIGHT OUTER JOIN 关系2 USING (同名等值连接属性列表)
- 全外连接：关系1 FULL OUTER JOIN 关系2 USING (同名等值连接属性列表)
- 每个同名连接属性只保留一个副本

例 (同名属性等值左外连接)

- ① 查询没有选课的学生的学号和姓名

```
SELECT Student.Sno, Sname
FROM Student LEFT OUTER JOIN SC USING (Sno)
WHERE Cno IS NULL;
```


自然外连接

- 自然左外连接: 关系1 NATURAL LEFT OUTER JOIN 关系2
- 自然右外连接: 关系1 NATURAL RIGHT OUTER JOIN 关系2
- 自然全外连接: 关系1 NATURAL FULL OUTER JOIN 关系2

例 (自然左外连接)

- ① 查询没有选课的学生的学号和姓名

```
SELECT Student.Sno, Sname
FROM Student NATURAL LEFT OUTER JOIN SC
WHERE Cno IS NULL;
```

课堂研讨: 表和自身连接

消除同名表的歧义

例 (表和自身连接)

- ① 查询和 Elsa 在同一个系学习的学生的学号和姓名

```
SELECT S2.Sno, S2.Sname
FROM Student AS S1 JOIN Student AS S2
ON S1.Sdept = S2.Sdept AND S1.Sno != S2.Sno
WHERE S1.Sname = 'Elsa';
```

习题: 连接查询

内连接

- ① What manufactures make laptops with a hard disk of at least 100GB?
- ② What PC models with a price less than \$500 does the manufacturer A produce?
- ③ Find the manufacturers of PC's with at least three different speeds

表和自身连接

- ① Find those pairs of PC models that have both the same speed and RAM (a pair should be listed only once)
- ② Find the model numbers of all printers that are cheaper than the printer model 3002
- ③ Find those hard-disk sizes that occur in two or more PC's

外连接

- ① ** Find the PC model with the highest available speed

3.3.5 嵌套查询

嵌套查询 (Nested Query)

SELECT 查询块 SELECT 查询块

子查询 (Subquery) 的类型

- 独立子查询
- 相关子查询

独立子查询 (Independent Subquery)

子查询独立于外层查询，可独立执行

例 (独立子查询)

- ① 查询和 Elsa 在同一个系学习的学生的学号和姓名 (含 Elsa)。似曾相识?

```
SELECT Sno, Sname FROM Student
WHERE Sdept = (SELECT Sdept FROM Student WHERE Sname = 'Elsa');
```

相关子查询 (Correlated Subquery)

子查询依赖于外层查询，不可独立执行

例 (相关子查询)

- ① 查询和 Elsa 在同一个系学习的学生的学号和姓名 (含 Elsa)。似曾相识?

```
SELECT Sno, Sname FROM Student AS S
WHERE EXISTS
  (SELECT * FROM Student AS T
   WHERE T.Sname = 'Elsa' AND T.Sdept = S.Sdept);
```

嵌套查询的写法

- ① 集合元素判断条件中使用子查询：使用 **[NOT] IN**
- ② 比较条件中使用子查询：使用 **比较运算符**
- ③ 结果存在性测试条件中使用子查询：使用 **[NOT] EXISTS**
- ④ 子查询结果集作为派生表：在FROM子句中使用
- ⑤ 子查询结果集作为临时表：使用 **WITH**

嵌套查询的写法 1：集合元素判断条件中使用子查询

表达式 **[NOT] IN** (子查询)

- 判断表达式的值是否（不）属于子查询结果集

例 (嵌套查询的写法 1)

- ① 查询和 Elsa 在同一个系学习的学生的学号和姓名 (含 Elsa)

```
SELECT Sno, Sname FROM Student
WHERE Sdept IN (SELECT Sdept FROM Student WHERE Sname = 'Elsa');
```

- ② 查询选修了 Database Systems 课程的学生的学号和姓名

```
SELECT Sno, Sname FROM Student
WHERE Sno IN (SELECT Sno FROM SC WHERE Cno IN
              (SELECT Cno FROM Course WHERE Cname = 'Database Systems'));
```

该嵌套查询既不易读，也不高效。有更好的做法吗？

嵌套查询的写法 1：集合元素判断条件中使用子查询

无查询优化：先执行子查询，后执行父查询

有查询优化：可能优化成连接查询

例 (嵌套查询的执行计划)

- ① 查询和 Elsa 在同一个系学习的学生的学号和姓名 (含 Elsa)

```
EXPLAIN SELECT Sno, Sname FROM Student
WHERE Sdept IN (SELECT Sdept FROM Student WHERE Sname = 'Elsa');
```

- ② 查询选修了 Database Systems 课程的学生的学号和姓名

```
EXPLAIN SELECT Sno, Sname FROM Student
WHERE Sno IN (SELECT Sno FROM SC WHERE Cno IN
              (SELECT Cno FROM Course WHERE Cname = 'Database Systems'));
```

嵌套查询的写法 2：比较条件中使用子查询

表达式 **比较运算符 [ALL|ANY|SOME]** (子查询)

- 将表达式的值与子查询的结果进行比较
- ALL：当表达式的值与子查询的任意结果都满足比较条件时，返回真；否则，返回假
- ANY或SOME：当表达式的值与子查询的某个结果满足比较条件时，返回真；否则，返回假
- 如果子查询结果集仅包含单个值，则可以省略ALL、ANY或SOME

例 (嵌套查询的写法 2)

- ① 查询和 Elsa 在同一个系学习的学生的学号和姓名 (含 Elsa)

```
SELECT Sno, Sname FROM Student
WHERE Sdept = (SELECT Sdept FROM Student WHERE Sname = 'Elsa');
```

嵌套查询的写法 2：比较条件中使用子查询

表达式 比较运算符 [ALL|ANY|SOME] (子查询)

例 (嵌套查询的写法 2)

- 1 查询年龄最大的学生的学号 (第 2 次出现, 上次出现是什么时候?)

```
SELECT Sno FROM Student
WHERE Sage = (SELECT MAX(Sage) FROM Student);
```

- 2 查询比计算机系全体学生年龄都大的学生的学号

```
SELECT Sno FROM Student
WHERE Sage > ALL (SELECT Sage FROM Student WHERE Sdept = 'CS');
```

- 3 查询学生平均年龄比全校学生平均年龄大的系

```
SELECT Sdept FROM Student
GROUP BY Sdept
HAVING AVG(Sage) > (SELECT AVG(Sage) FROM Student);
```

嵌套查询的写法 2：比较条件中使用子查询

无查询优化：先执行子查询，后执行父查询

有查询优化：可能优化成连接查询

例 (嵌套查询的执行计划)

- 1 查询和 Elsa 在同一个系学习的学生的学号和姓名 (含 Elsa)

```
EXPLAIN SELECT Sno, Sname FROM Student
WHERE Sdept = (SELECT Sdept FROM Student WHERE Sname = 'Elsa');
```

嵌套查询的写法 3：结果存在性测试条件中使用子查询

[NOT] EXISTS (子查询)

- 判断子查询是否有 (无) 结果

例 (EXISTS 谓词)

- 1 执行下面的示例查询

```
SELECT * FROM Student
WHERE EXISTS (SELECT 'This subquery has a result');
```

嵌套查询的写法 3：结果存在性测试条件中使用子查询

[NOT] EXISTS (子查询)

- 判断子查询是否有 (无) 结果

例 (嵌套查询的写法 3)

- 1 查询和 Elsa 在同一个系学习的学生的学号和姓名 (含 Elsa)

```
SELECT Sno, Sname FROM Student AS S
WHERE EXISTS
  (SELECT * FROM Student AS T
   WHERE T.Sname = 'Elsa' AND T.Sdept = S.Sdept);
```

只需判断子查询是否有结果，并不需要做投影，故子查询用 SELECT * 即可

嵌套查询的写法 3: 结果存在性测试条件中使用子查询

先执行父查询，执行父查询过程中执行子查询

例 (嵌套查询的执行计划)

- ① 查询和 Elsa 在同一个系学习的学生的学号和姓名 (含 Elsa)

```
EXPLAIN SELECT Sno, Sname FROM Student AS S
WHERE EXISTS
  (SELECT * FROM Student AS T
   WHERE T.Sname = 'Elsa' AND T.Sdept = S.Sdept);
```

- ① 从 S 中取出下一条元组 t，获得元组的 Sdept 属性值，用变量 S.Sdept 表示
- ② 将 S.Sdept 传给子查询，并执行子查询
- ③ 若子查询有结果，则元组 t 满足查询条件，投影到 Sno 和 Sname 属性上

嵌套查询的写法 4: 子查询结果集作为派生关系

FROM (子查询) AS 表名

- 将子查询结果集作为派生表 (Derived Table)，在父查询中使用
- 子查询必须是独立子查询
- 派生表必须重命名

例 (嵌套查询的写法 4)

- ① 查询选修了 2 门以上课程的学生的学号和选课数

```
SELECT Sno, T.Amt FROM
  (SELECT Sno, COUNT(*) AS Amt FROM SC GROUP BY Sno) AS T
WHERE T.Amt >= 2;
```

第 2 次出现，上一次是怎么做的？还能怎么做？

嵌套查询的写法 5: 子查询结果集作为临时表

WITH 临时表名(属性列表) AS (子查询)

- 放在查询语句开始处，将子查询定义为临时表
- 临时表只在包含 WITH 子句的查询中有效

例 (嵌套查询的写法 5)

- ① 查询选修了 2 门以上课程的学生的学号和选课数

```
WITH T AS (SELECT Sno, COUNT(*) AS Amt FROM SC GROUP BY Sno)
SELECT Sno, Amt FROM T WHERE Amt >= 2;
```

第 3 次出现，上一次是怎么做的？还能怎么做？

嵌套查询的执行

- 嵌套查询的执行计划通常是由 DBMS 的查询优化器决定的
- 查询优化器可能会将一个嵌套查询“扁平化”为一个执行效率更高的连接查询
- 在 PostgreSQL 和 openGauss 中，可以通过设置配置参数 from_collapse_limit = 1 来强制查询优化器采用嵌套 SQL 语句中给出子查询执行顺序

习题：嵌套查询

用IN

- 1 Find the manufacturers that sell laptops but not PC's

用比较运算符

- 1 Find the model numbers of all printers that are cheaper than the printer model 3002
- 2 Find the PC model with the highest available speed

用EXISTS

- 1 Find the model numbers of all printers that are cheaper than the printer model 3002
- 2 Find the PC model with the highest available speed

用派生表或WITH

- 1 Find the manufacturers of PC's with at least three different speeds

3.4 SQL 数据更新

SQL 数据更新

- 插入元组
- 删除元组
- 修改元组

3.4.1 插入元组

插入元组

INSERT

- 插入语句中列出的元组
- 插入查询结果

插入语句中列出的元组

INSERT INTO ... VALUES ...

例 (插入语句中列出的元组)

- ① 按表中属性的顺序列出元组的全部属性值

```
INSERT INTO Student VALUES
('MA-001', 'Abby', 'F', 18, 'Math'),
('MA-002', 'Cindy', 'F', 19, 'Math');
```

- ② 按语句中指定的属性列表给出元组的属性值

```
INSERT INTO Student (Sno, Sname, Sage, Ssex) VALUES
('MA-001', 'Abby', 18, 'F'),
('MA-002', 'Cindy', 19, 'F');
```

插入查询结果

INSERT INTO ... SELECT语句

例 (插入查询结果)

- ① 将计算机系学生的学号、姓名、选课数、平均分插入表 CS_Grade(Sno, Sname, Amt, AvgGrade) 中

```
INSERT INTO CS_Grade
SELECT Sno, Sname, COUNT(*), AVG(Grade)
FROM Student NATURAL JOIN SC
WHERE Sdept = 'CS'
GROUP BY Sno, Sname;
```

基于查询结果创建表

CREATE TABLE ... AS SELECT ...

例 (基于查询结果创建表)

- ① 创建表 CS_Grade, 存储计算机系学生的学号、姓名、选课数、平均分

```
CREATE TABLE CS_Grade AS
SELECT Sno, Sname, COUNT(*) AS Amt, AVG(Grade) AS AvgGrade
FROM Student NATURAL JOIN SC
WHERE Sdept = 'CS'
GROUP BY Sno, Sname;
```

3.4.2 删除元组

删除元组

DELETE FROM 表名 WHERE ...

例 (删除元组)

- ① 删除学号为 MA-002 的学生元组

```
DELETE FROM Student WHERE Sno = 'MA-002';
```

- ② 删除选修了 1002 号课程的学生元组

```
DELETE FROM Student WHERE EXISTS  
(SELECT * FROM SC WHERE SC.Sno = Student.Sno AND Cno = '1002');
```

还有其他写法吗?

3.4.3 修改元组

修改元组

UPDATE 表名 SET ... WHERE ...

例 (删除元组)

- ① 将学号为 MA-002 的学生的年龄修改为 20 岁

```
UPDATE Student SET Sage = 20 WHERE Sno = 'MA-002';
```

- ② 将所有学生的年龄增加 1 岁

```
UPDATE Student SET Sage = Sage + 1;
```

- ③ 将计算机系学生的 1002 号课程成绩重置为 0

```
UPDATE SC SET Grade = 0 WHERE Cno = '1002' AND EXISTS  
(SELECT * FROM Student  
WHERE Sdept = 'CS' AND Student.Sno = SC.Sno);
```

还有其他写法吗?

3.4.4 完整性约束检查

实体完整性约束检查

检查时机

- 插入元组时
- 修改元组的主键属性值时

处理方法

- 如果插入或修改的元组的某个主键属性值为空，则拒绝插入或修改元组
- 如果插入或修改的元组的主键值与某个现有元组相同，则拒绝插入或修改元组

用户定义完整性约束检查

检查时机

- 插入元组时
- 修改元组时

处理方法

- 若更新后的数据库不满足用户定义完整性约束，则拒绝插入或修改元组

参照完整性检查

有四种情况可能会破坏参照完整性

情况 1

在参照关系（如 SC）中插入一条元组时，如果该元组的外键（如 Sno）的值在被参照关系（如 Student）的主键（如 Sno）值集合中不存在，即悬空（dangling），则拒绝插入该元组

SC		
Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	1002	95
CS-001	3006	90
CS-002	3006	80
MA-001	1002	
MA-003	1002	

拒绝插入

Student				
Sno	Sname	Ssex	Sage	Sdept
PH-001	Nick	M	20	Physics
CS-001	Elsa	F	19	CS
CS-002	Ed	M	19	CS
MA-001	Abby	F	18	Math
MA-002	Cindy	F	19	Math

参照完整性检查

情况 2

修改参照关系（如 SC）的一条元组时，如果造成该元组的外键（如 Sno）的值在被参照关系（如 Student）的主键（如 Sno）值集合中不存在，则拒绝修改该元组

SC		
Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	1002	95
CS-001	3006	90
CS-002	3006	80
MA-001	1002	
MA-003		

拒绝修改

Student				
Sno	Sname	Ssex	Sage	Sdept
PH-001	Nick	M	20	Physics
CS-001	Elsa	F	19	CS
CS-002	Ed	M	19	CS
MA-001	Abby	F	18	Math
MA-002	Cindy	F	19	Math

参照完整性检查

情况 3

从被参照关系（如 Student）中删除一个元组，如果造成参照关系（如 SC）中某些元组的外键（如 Sno）的值在被参照关系（如 Student）的主键（如 Sno）值集合中不存在，则采取下列处理方法之一：

- 1 如果参照关系（如 SC）中没有任何元组的外键值与被参照关系（如 Student）中待删除元组的主键值对应，则删除该元组；否则，拒绝删除该元组
- 2 级联（cascade）删除参照关系（如 SC）中相关联的元组
- 3 将参照关系（如 SC）中相关联元组的外键（如 Sno）值置为空

SC		
Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	1002	95
CS-001	3006	90
CS-002	3006	80
MA-001	1002	
MA-003		

Student				
Sno	Sname	Ssex	Sage	Sdept
PH-001	Nick	M	20	Physics
CS-001	Elsa	F	19	CS
CS-002	Ed	M	19	CS
MA-001	Abby	F	18	Math
MA-002	Cindy	F	19	Math

拒绝删除

参照完整性检查

情况 4

修改被参照关系（如 Student）中一个元组，如果造成参照关系（如 SC）中某些元组的外键（如 Sno）的值在被参照关系（如 Student）的主键（如 Sno）值集合中不存在，则采取下列处理方法之一：

- 1 如果参照关系（如 SC）中没有任何元组的外键值与被参照关系（如 Student）中待删除元组的主键值对应，则修改该元组；否则，拒绝修改该元组
- 2 级联（cascade）修改参照关系（如 SC）中相关联的元组
- 3 将参照关系（如 SC）中相关联元组的外键（如 Sno）值置为空

SC		
Sno	Cno	Grade
PH-001	1002	92
PH-001	2003	85
PH-001	3006	88
CS-001	1002	95
CS-001	3006	90
CS-002	3006	80
MA-001	1002	
MA-003		

Student				
Sno	Sname	Ssex	Sage	Sdept
PH-001	Nick	M	20	Physics
CS-001	Elsa	F	19	CS
CS-002	Ed	M	19	CS
MA-001	Abby	F	18	Math
MA-003				
MA-002	Cindy	F	19	Math

拒绝修改

设置参照完整性检查方法

在FOREIGN KEY子句的末尾加上

ON DELETE NO ACTION | RESTRICT | CASCADE | SET NULL | SET DEFAULT
ON UPDATE NO ACTION | RESTRICT | CASCADE | SET NULL | SET DEFAULT

- RESTRICT或NO ACTION：拒绝删除或修改，缺省处理方式
- CASCADE：级联删除或修改
- SET NULL：置为空值
- SET DEFAULT：置为被参照属性的缺省值

例 (参照完整性检查)

```
FOREIGN KEY (Sno) REFERENCES Student(Sno)
ON DELETE RESTRICT
ON UPDATE RESTRICT
```

视图

3.5 视图

视图 = 外模式 = 虚拟表

创建视图

CREATE VIEW

例 (创建视图)

❶ 为选修了 3006 号课的计算机系的学生创建视图，列出学号、姓名和成绩

```
CREATE VIEW CS_Student_on_3006 AS
SELECT Sno, Sname, Grade
FROM Student NATURAL JOIN SC
WHERE Sdept = 'CS' AND Cno = '3006';
```

删除视图

DROP VIEW

例 (删除视图)

❶ 删除视图CS_Student_on_3006

```
DROP VIEW CS_Student_on_3006;
```

查询视图

例 (查询视图)

- ① 查询计算机系通过了 3006 号课考核的学生
- ```
| SELECT Sno, Sname FROM CS_Student_on_3006 WHERE Grade >= 60;
```

视图上的查询会被 DBMS 转换为表上的查询

更新视图

视图上不是什么更新都能做

例 (更新视图)

- ① 将学号为 CS-001 的学生的 3006 号课成绩置为 100
- ```
| UPDATE CS_Student_on_3006 SET Grade = 100 WHERE Sno = 'CS-001';
```

视图上的更新会被 DBMS 转换为表上的更新

探究式学习：物化视图 (Materialized View)

借助 LLM 学习

- 什么是物化视图?
- 物化视图与视图有什么区别?
- 物化视图有什么优点?
- 如何创建, 删除, 修改, 更新物化视图?

3.6 索引

加速数据访问的数据结构

CREATE INDEX

例 (创建索引)

- 在 Student 表的属性列表 (Sname) 上创建索引
- ```
CREATE INDEX Idx_Student_Sname ON Student (Sname);
```

索引|键具有唯一性的索引

CREATE UNIQUE INDEX

例 (创建唯一索引)

- 在 Student 表的属性列表 (Sname) 上创建唯一索引
- ```
CREATE UNIQUE INDEX Idx_Student_Sname ON Student (Sname);
```

DROP INDEX

例 (删除索引)

- 删除 Student 表上的索引Idx_Student_Sname
- ```
DROP INDEX Idx_Student_Sname;
```

## 教学内容

- ① 3.1 SQL 标准数据类型
- ② 3.2 SQL 数据定义
  - 3.2.1 创建表
  - 3.2.2 删除表
  - 3.2.3 修改表模式
- ③ 3.3 SQL 数据查询
  - 3.3.1 单表查询
  - 3.3.2 集合查询
  - 3.3.3 分组查询
  - 3.3.4 连接查询
  - 3.3.5 嵌套查询
- ④ 3.4 SQL 数据更新
  - 3.4.1 插入元组
  - 3.4.2 删除元组
  - 3.4.3 修改元组
  - 3.4.4 完整性约束检查

## ⑤ 3.5 视图

邹兆年 (CS@HIT)

第 3 章：关系数据库标准语言 SQL

2026 春

117 / 121

## 习题 II

- ⑨ 若关系  $R$  的属性  $A$  被声明为 UNIQUE, 则 SQL 语句  $\text{SELECT COUNT}(A) \text{ FROM } R$  的结果是  $|R|$
- ⑩ 设  $R(a, b)$  和  $S(a)$  是两个关系, 将关系代数表达式  $R \div S$  用 SQL 语句表示
- ⑪ 在 openGauss 或 MySQL 上创建 Product 数据库 (Database Systems The Complete Book - Exercise 2.4.1), 然后使用 SQL 表达下列数据库查询与更新, 并在 DBMS 上验证
  - ① Find the manufacturers that sell laptops but not PC's. (使用集合差运算)
  - ② Find the manufacturers that sell laptops but not PC's. (使用含有 IN 的嵌套查询)
  - ③ Find the manufacturers that sell laptops but not PC's. (使用含有 EXISTS 的嵌套查询)
  - ④ Find the model numbers of all printers that are cheaper than the printer model 3002. (使用内连接查询)
  - ⑤ Find the model numbers of all printers that are cheaper than the printer model 3002. (使用含有比较运算符的嵌套查询)
  - ⑥ Find the model numbers of all printers that are cheaper than the printer model 3002. (使用含有 EXISTS 的嵌套查询)
  - ⑦ Find the PC model with the highest available speed. (使用外连接查询)

邹兆年 (CS@HIT)

第 3 章：关系数据库标准语言 SQL

2026 春

119 / 121

## 习题 I

- ① 身份证号用 CHAR(13) 和 VARCHAR(13) 哪种类型表示更合适?
- ② 什么情况下需要使用 UNION ALL? 什么情况下使用 UNION?
- ③ MySQL 不支持集合交操作 INTERSECT, 如何实现集合交操作?
- ④ MySQL 不支持集合差操作 EXCEPT, 如何实现集合差操作?
- ⑤ 为什么分组查询语句的目标属性只能包含分组属性和聚集函数?
- ⑥ 相关子查询和不相关子查询有何区别?
- ⑦ 为什么子查询中通常不使用 DISTINCT 和 ORDER BY?
- ⑧ 视图和基本表有什么区别?
- ⑨ 判断对错
  - ① SQL 语句  $\text{DELETE FROM TABLE } R$  从数据库中删除关系  $R$
  - ② 将属性声明为 PRIMARY KEY 和 UNIQUE NOT NULL 作用是一样的
  - ③ ORDER BY A, B DESC 将查询结果按照属性  $A$  和  $B$  的值降序排列
  - ④ SQL 语句  $\text{SELECT } A \text{ FROM } R$  与关系代数表达式  $\Pi_A(R)$  的结果相同

邹兆年 (CS@HIT)

第 3 章：关系数据库标准语言 SQL

2026 春

118 / 121

## 习题 III

- ⑧ Find the PC model with the highest available speed. (使用含有 IN 的嵌套查询)
- ⑨ Find the PC model with the highest available speed. (使用含有 = 的嵌套查询)
- ⑩ Find the PC model with the highest available speed. (使用含有 >= 的嵌套查询)
- ⑪ Find the PC model with the highest available speed. (使用含有 EXISTS 的嵌套查询)
- ⑫ Find the manufacturers of PC's with at least three different speeds. (使用内连接查询)
- ⑬ Find the manufacturers of PC's with at least three different speeds. (使用分组查询)
- ⑭ Find the manufacturers of PC's with at least three different speeds. (使用派生关系)
- ⑮ Decrease the price of all PC's made by maker A by 10%. (使用含有 = 的更新条件)
- ⑯ Decrease the price of all PC's made by maker A by 10%. (使用含有 IN 的更新条件)
- ⑰ Decrease the price of all PC's made by maker A by 10%. (使用含有 EXISTS 的更新条件)
- ⑱ 前一题 (g) 中的查询可以用多种 SQL 语句表示。尝试从语句的易读性和执行效率两方面对 (g)-(k) 的 SQL 语句进行分析和比较。在做效率分析时, 我们假定每个关系上只有主索引, 而没有其他索引 (请自学索引的概念)。

邹兆年 (CS@HIT)

第 3 章：关系数据库标准语言 SQL

2026 春

120 / 121

## 探究式学习

- 使用 LLM 生成例题和习题中的查询
- 与 LLM 探讨如何判断查询语句的写法是否正确?
- 与 LLM 探讨什么是查询改写? 如何进行查询改写?

## 致谢

2026 版以前

- 感谢孔美豪、赖昕、王智义、王润哲、宋明阳 (1190200604)、詹儒彦 (1190202307)、陆任驰 (1183200212) 同学发现课件中的问题
- 感谢禹棋赢同学补充了一个知识点
- 感谢龚利锋、王梓宣同学提供课堂练习题笔记